UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MENDOZA

Ingeniería y Calidad de Software

Trabajo Práctico N.º 6

Testing Automatizado

GRUPO N.º 4 – 4K10

MÓDULO 1 (Autenticación) Y MÓDULO 4 (Booking Público)

Integrantes:

Barchiesi Santino – 50791

Estudillo Máximo – 50013

Navarro Matías – 51006

Repositorio: https://github.com/maxidoe2/agendaya-tp6-grupo4

Mendoza, octubre de 2026

Índice

1. Repositorio y organización del equipo 3
1.1 Responsabilidades individuales 3
2. Tarea A – Frontend mínimo 4
2.1 Descripción 4
2.2 Flujos implementados 4
2.3 Criterios de aceptación 7
3. Tarea B – Tests E2E con Cypress 8
3.1 Organización 8
3.2 Código de los tests 8
3.3 Ejecución y evidencia 12
3.4 Análisis de resultados 17
4. Tarea C – Tests unitarios con IA 19
4.1 Organización 19
4.2 Uso de IA: proceso general 19
4.3 Tests y uso de IA por integrante 20
4.4 Evidencia de ejecución 27
5. Reflexión estructurada 29
5.1 Sobre la trazabilidad 29
5.2 Sobre el valor de testear 29
5.3 Sobre el uso de IA 29
6. Lecciones aprendidas 30

1. Repositorio y organización del equipo

Repositorio: https://github.com/maxidoe2/agendaya-tp6-grupo4

Contiene el frontend, el servidor, la lógica de negocio, los tests E2E y unitarios, las evidencias y este informe. El `README.md` explica cómo levantar la aplicación y ejecutar cada tipo de test. El historial tiene 8 commits que muestran el avance del trabajo: base del TP5 → extracción de la lógica de negocio → frontend testeable → tests E2E → tests unitarios → documentación.

La aplicación parte de la construida en el TP5 (https://github.com/maxidoe2/agendaya-tp5-grupo4).

1.1 Responsabilidades individuales

Cada integrante es responsable de al menos un test E2E y cinco tests unitarios, sobre al menos dos funciones distintas:

Integrante	Tests E2E (cypress/e2e/)	Tests unitarios (tests/unit/)
Barchiesi Santino	m01-registro-exitoso (1) m01-login-logout (2)	m01-email (8) m01-seguridad (8)
Estudillo Máximo	m01-registro-errores (2) m04-reserva-errores (3)	m01-password-registro (8) m04-validacion-conflictos (10)
Navarro Matías	m04-reserva-exitosa (2)	m04-slots-antelacion (11)
Total	10 tests	45 tests

El frontend mínimo es un entregable grupal y los tres integrantes conocen su estructura.

2. Tarea A – Frontend mínimo

2.1 Descripción

Se partió de la aplicación del TP5: Node.js con Express, pantallas en HTML, CSS y JavaScript vanilla, datos en un archivo JSON y correos simulados que se registran en `data/correos.log`. El enunciado permite prescindir del backend, pero se mantuvo porque ya existía y permite verificar estados reales del sistema: reservas creadas y correos disparados. La aplicación se reorganizó para que fuera testeable:

- **Lógica de negocio separada de la interfaz.** En el TP5 las reglas estaban dentro de `server.js` y de los scripts de cada pantalla. Se movieron a `src/logica/validaciones.js` (M01) y `src/logica/reservas.js` (M04). Son funciones puras que se cargan tanto en Node como en el navegador: la validación que corre en la pantalla es la misma que prueba Jest y que vuelve a aplicar el servidor.
- **Selección real de fecha y hora.** En el TP5 los horarios estaban fijos al 15/09/2026, una fecha que ya pasó. Ahora el invitado elige un día y la agenda se calcula a partir de la disponibilidad: lunes a viernes, de 9 a 17 h, turnos de 60 minutos y 2 horas de antelación mínima. Como la disponibilidad corresponde al módulo M02, se fijó como configuración.
- **El reloj se inyecta.** Las funciones que dependen de la hora actual reciben `ahora` como parámetro, por lo que los tests no dependen del día en que se ejecutan.
- **Atributos `data-cy`** en todos los elementos interactivos y en los mensajes que verifican los tests: campos (`email-input`, `fecha-input`), botones (`login-button`, `submit-booking`), turnos (`slot-1000`) y mensajes (`alerta-error`, `alerta-exito`, `error-<campo>`, `booking-confirmation`).

2.2 Flujos implementados

Módulo	Flujo	Pantalla	Requerimiento
M01	Registro de usuario	/registro.html	AYA-M01-RF-01
M01	Login y logout	/ → /dashboard.html → Cerrar sesión	AYA-M01-RF-04
M04	Seleccionar fecha y hora	/reservar.html (pasos 1 y 2)	AYA-M04-RF-03
M04	Completar formulario y confirmar reserva	/reservar.html (pasos 3 y 4)	AYA-M04-RF-03 y RF-05
M04 (opcional)	Manejo de horario no disponible	/reservar.html	AYA-M04-RF-05

También se conservan del TP5 la recuperación y el restablecimiento de contraseña, con sus atributos `data-cy`.

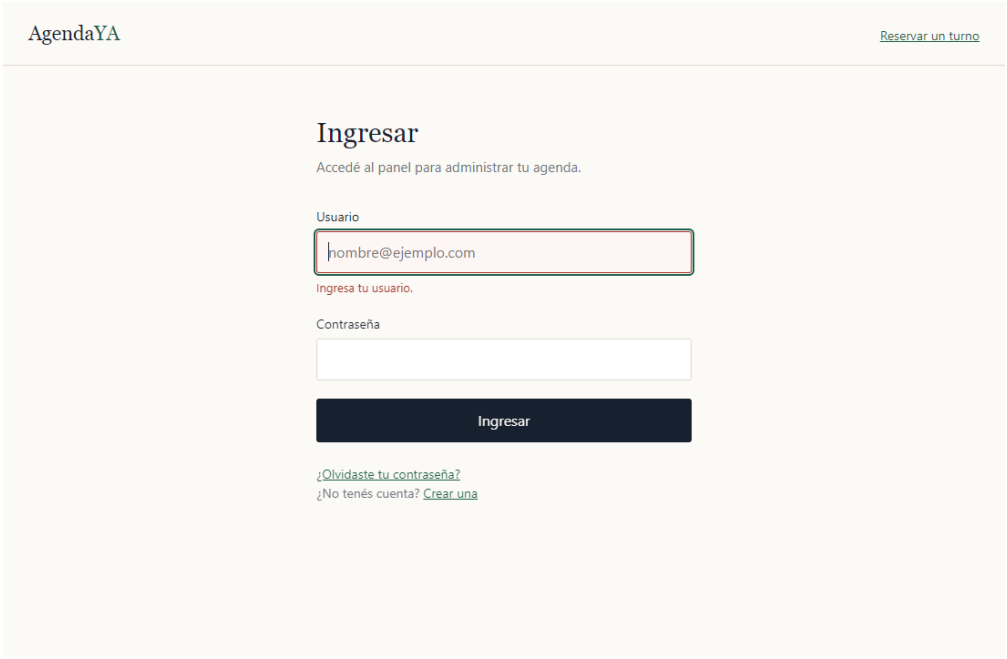


Figura 1 – Login: al enviar el formulario vacío se muestra el error debajo del campo.

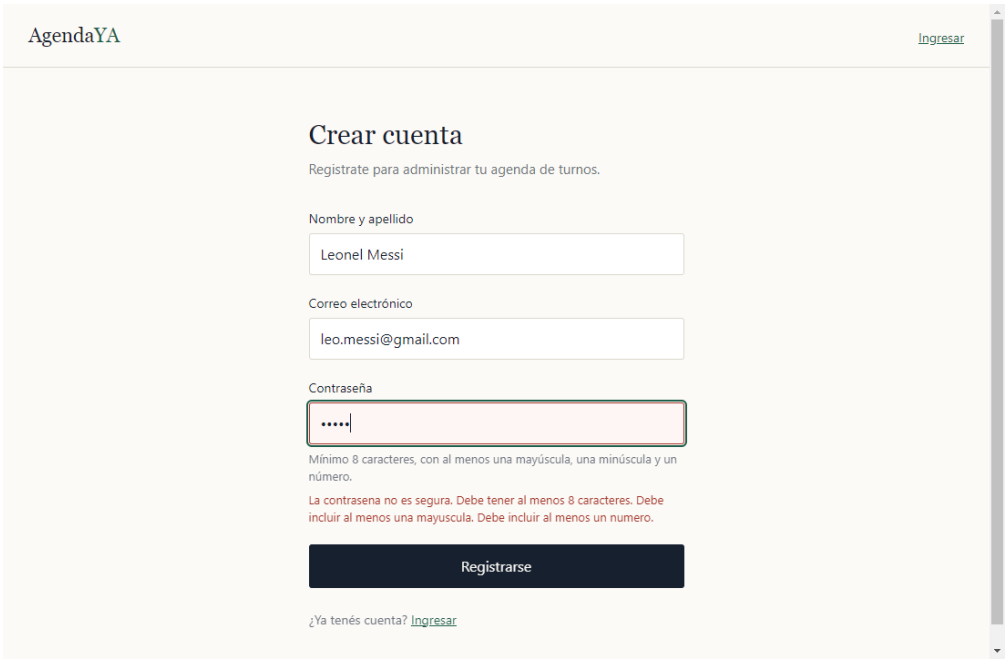


Figura 2 – Registro: la contraseña no cumple los requisitos de seguridad y se informa cada regla.

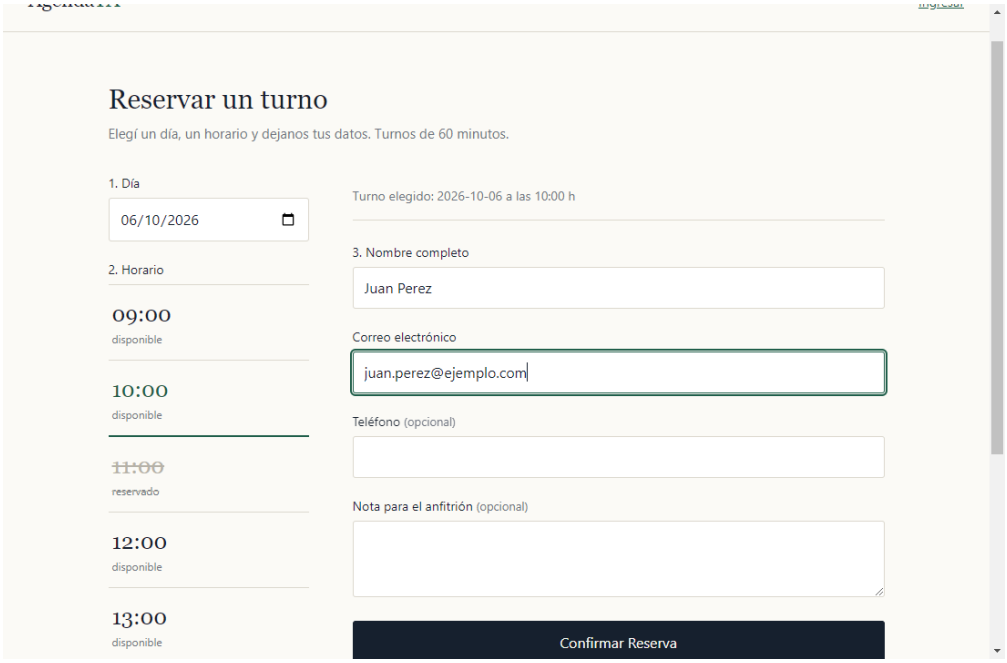


Figura 3 – Reserva: día y horario elegidos; el turno de las 11:00 ya está reservado.

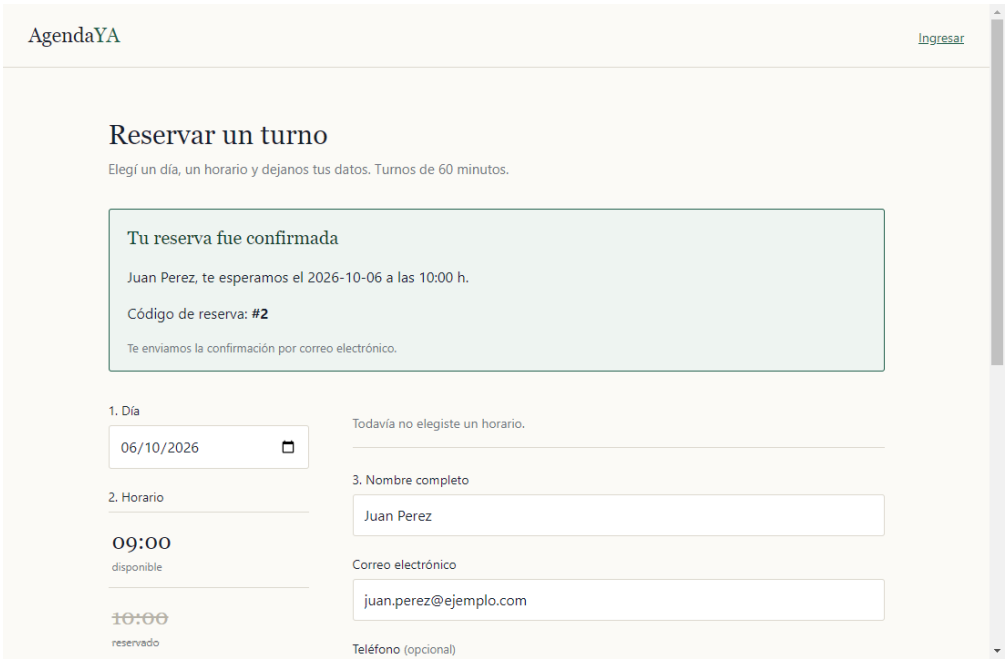


Figura 4 – Reserva confirmada con su código.

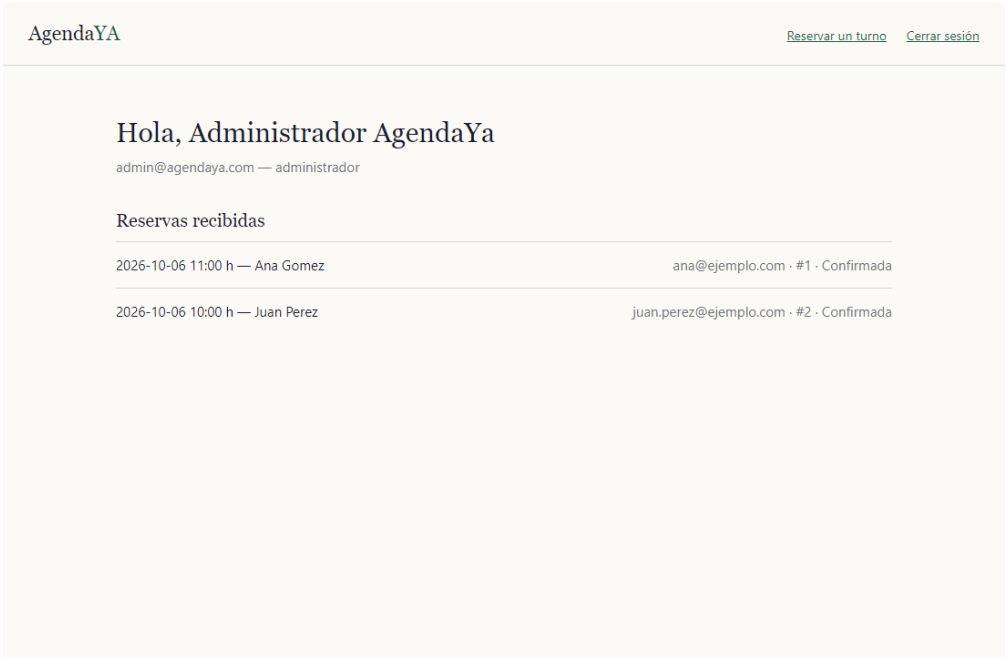


Figura 5 – Panel del administrador con las reservas recibidas y el botón Cerrar sesión.

2.3 Criterios de aceptación

Criterio	Cumplimiento
Los dos flujos obligatorios son navegables sin errores	Sí. Los recorren completos los tests E2E (10/10 aprobados).
Todos los elementos interactivos tienen data-cy	Sí. Se verificó buscando en todas las pantallas input, button, a, textarea y form sin el atributo (resultado vacío). Los turnos generados por JavaScript reciben data-cy="slot-HHMM".
Los formularios validan el campo vacío con un mensaje visible	Sí. Login, registro, recuperación y reserva muestran el motivo debajo del campo (error-<campo>). Lo verifican m01-registro-errores y m04-reserva-errores.
Al completar un flujo se muestra una confirmación visible	Sí. Registro: "Tu cuenta fue creada correctamente."; logout: "Cerraste sesión correctamente."; reserva: panel booking-confirmation con el código.
Repositorio Git con al menos 3 commits descriptivos	Sí. 8 commits, cada uno con una descripción del cambio y su motivo.

3. Tarea B – Tests E2E con Cypress

3.1 Organización

- `baseUrl: http://localhost:3000` se configura una sola vez en `cypress.config.js` y los tests usan rutas relativas.
- Antes de cada test se reinicia la base con `cy.task("reiniciarBase")`: cada test parte de un estado conocido y no depende de los demás ni del orden de ejecución.
- Para verificar efectos que no se ven en pantalla se usan `cy.task("leerCorreos")` (correos disparados), `cy.request("/api/reservas")` (reservas guardadas) y `cy.intercept` (pedidos al servidor y códigos de respuesta).
- Los tests de reserva calculan el próximo día hábil: con una fecha fija, el test quedaría en el pasado y el sistema, correctamente, la rechazaría.
- Al final de cada test se guarda una captura del runner (aplicación y registro de comandos).

Integrante	Archivo	Tests	Variante
Barchiesi Santino	m01-registro-exitoso	Registro de un administrador nuevo	Happy path
Barchiesi Santino	m01-login-logout	Login + logout + panel protegido Contraseña incorrecta	Happy path Datos inválidos
Estudillo Máximo	m01-registro-errores	Formulario vacío Correo ya registrado	Datos inválidos Estado del sistema
Estudillo Máximo	m04-reserva-errores	Horario tomado por otro invitado Falta el correo Día no hábil	Estado del sistema Datos inválidos Caso borde
Navarro Matías	m04-reserva-exitosa	Fecha, hora y confirmación Clicks múltiples	Happy path Estado del sistema

3.2 Código de los tests

Barchiesi Santino – Registro exitoso

Archivo: `cypress/e2e/m01-registro-exitoso.cy.js`

```
/**
 * M01 - Autenticacion | Flujo obligatorio: Registro de usuario (happy path)
 * Responsable: Barchiesi Santino
 * Trazabilidad: AYA-M01-RF-01 / CP-003
 */
describe("AgendaYA - M01 Autenticacion: registro exitoso", () => {
  beforeEach(() => {
    cy.visit("/registro.html");
  });

  it("registra un administrador nuevo y lo deja dentro del panel", () => {
    // Arrange: completar el formulario con datos validos y un correo nuevo
    cy.get('[data-cy="nombre-input"]').type("Leonel Messi");
    cy.get('[data-cy="email-input"]').type("leo.messi@gmail.com");
    cy.get('[data-cy="password-input"]').type("Messimessi12");


    // Act: enviar el registro
    cy.get('[data-cy="registro-button"]').click();


    // Assert: se crea la cuenta y se accede al panel con la sesion del usuario nuevo
    cy.url().should("include", "/dashboard");
    cy.get('[data-cy="alerta-exito"]')
      .should("be.visible")
```

```
      .and("contain", "Tu cuenta fue creada correctamente.");
    cy.get('[data-cy="dashboard-welcome"]').should("have.text", "Hola, Leonel Messi");
    cy.get('[data-cy="usuario-email"]').should("contain", "leo.messi@gmail.com");
  });
});
```

Barchiesi Santino – Login y logout

Archivo: cypress/e2e/m01-login-logout.cy.js

```
/**
 * M01 - Autenticacion | Flujo obligatorio: Login y logout
 * Responsable: Barchiesi Santino
 * Trazabilidad: AYA-M01-RF-04 / CP-001 y CP-002
 */
describe("AgendaYA - M01 Autenticacion: login y logout", () => {
  beforeEach(() => {
    cy.visit("/");
  });

  it("inicia sesion, entra al panel y al cerrar sesion pierde el acceso", () => {
    // Arrange: credenciales del administrador precargado
    cy.get('[data-cy="email-input"]').type("admin@agendaya.com");
    cy.get('[data-cy="password-input"]').type("Contrasena123!");

    // Act (1): iniciar sesion
    cy.get('[data-cy="login-button"]').click();

    // Assert (1): llega al panel con su usuario
    cy.url().should("include", "/dashboard");
    cy.get('[data-cy="dashboard-welcome"]').should("have.text", "Hola, Administrador AgendaYa");

    // Act (2): cerrar sesion
    cy.get('[data-cy="logout-button"]').click();

    // Assert (2): vuelve al login con confirmacion y el panel queda protegido
    cy.location("pathname").should("eq", "/");
    cy.get('[data-cy="alerta-exito"]').should("be.visible").and("have.text", "Cerraste sesión
correctamente.");
    cy.visit("/dashboard.html");
    cy.location("pathname").should("eq", "/");
  });

  it("con contrasena incorrecta no ingresa y limpia el campo de contrasena", () => {
    // Arrange: usuario existente con una contrasena equivocada
    cy.get('[data-cy="email-input"]').type("admin@agendaya.com");
    cy.get('[data-cy="password-input"]').type("ContrasenaInvalida123");

    // Act: intentar ingresar
    cy.get('[data-cy="login-button"]').click();

    // Assert: mensaje exacto de CP-002, campo vacio y sin acceso al panel
    cy.get('[data-cy="alerta-error"]')
      .should("be.visible")
      .and("have.text", "Contrasena incorrecta, intentelo de nuevo.");
    cy.get('[data-cy="password-input"]').should("have.value", "");
    cy.location("pathname").should("eq", "/");
  });
});
```

Estudillo Máximo – Registro con errores

Archivo: cypress/e2e/m01-registro-errores.cy.js

```
/**
 * M01 - Autenticacion | Flujo obligatorio: Registro de usuario (variantes de error)
 * Responsable: Estudillo Maximo
 * Trazabilidad: AYA-M01-RF-01 / CP-004
 */
describe("AgendaYA - M01 Autenticacion: registro con errores", () => {
  beforeEach(() => {
```

```
    cy.intercept("POST", "/api/registro").as("registro");
    cy.visit("/registro.html");
  });

  it("no envia el formulario vacio y explica que falta el nombre", () => {
    // Arrange: formulario sin completar (estado inicial de la pantalla)
    cy.get('[data-cy="nombre-input"]').should("have.value", "");

    // Act: intentar registrarse sin datos
    cy.get('[data-cy="registro-button"]').click();

    // Assert: error visible en el campo, sin pedido al servidor y sin salir de la pantalla
    cy.get('[data-cy="error-nombre"]').should("be.visible").and("have.text", "Ingresa tu nombre.");
    cy.get("@registro.all").should("have.length", 0);
    cy.url().should("include", "/registro.html");
  });

  it("rechaza un correo que ya esta registrado y no crea la cuenta", () => {
    // Arrange: ya existe una cuenta con ese correo (precondicion de CP-004)
    cy.request("POST", "/api/registro", {
      nombre: "Leonel Messi",
      email: "leo.messi@gmail.com",
      password: "Messimessi12",
    });
    cy.get('[data-cy="nombre-input"]').type("Leonel Messi");
    cy.get('[data-cy="email-input"]').type("leo.messi@gmail.com");
    cy.get('[data-cy="password-input"]').type("Messimessi13");

    // Act: intentar registrar el mismo correo otra vez
    cy.get('[data-cy="registro-button"]').click();

    // Assert: el servidor responde 409, se informa el motivo y no se entra al panel
    cy.wait("@registro").its("response.statusCode").should("eq", 409);
    cy.get('[data-cy="alerta-error"]')
      .should("be.visible")
      .and("have.text", "El correo ya existe en el sistema.");
    cy.get('[data-cy="error-email"]').should("be.visible");
    cy.url().should("include", "/registro.html");
  });
});
```

Estudillo Máximo – Reserva con errores

Archivo: `cypress/e2e/m04-reserva-errores.cy.js`

```
/**
 * M04 - Booking publico | Variantes de error del flujo de reserva:
 *   - por estado del sistema: horario no disponible (flujo opcional del modulo)
 *   - por datos invalidos: falta el correo electronico
 * Responsable: Estudillo Maximo
 * Trazabilidad: AYA-M04-RF-03 y AYA-M04-RF-05 / CP-008 / enunciado 3.1
 */
import { proximoDiaHabil, proximoSabado } from "../support/fechas";

describe("AgendaYA - M04 Booking: reserva con errores", () => {
  const dia = proximoDiaHabil();

  beforeEach(() => {
    cy.intercept("POST", "/api/reservas").as("crearReserva");
    cy.visit("/reservar.html");
  });

  it("informa que el horario ya fue reservado si otro invitado lo tomo antes", () => {
    // Arrange: el invitado elige las 15:00 y completa sus datos...
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1500"]').should("be.enabled").click();
    cy.get('[data-cy="nombre-input"]').type("Ana Gomez");
    cy.get('[data-cy="email-input"]').type("ana.gomez@ejemplo.com");
    // ...mientras tanto otro invitado reserva ese mismo turno desde otro dispositivo
    cy.request("POST", "/api/reservas", {
      nombre: "Otro Invitado",
      email: "otro@ejemplo.com",
      fecha: dia,
      hora: "15:00",
```

```
    });

    // Act: confirmar la reserva sobre un turno que ya no esta libre
    cy.get('[data-cy="submit-booking"]').click();

    // Assert: conflicto 409 con el mensaje del enunciado, agenda actualizada y
    // el turno sigue siendo del primer invitado
    cy.wait("@crearReserva").its("response.statusCode").should("eq", 409);
    cy.get('[data-cy="alerta-error"]')
      .should("be.visible")
      .and("have.text", "Este horario ya fue reservado. Por favor elegí otro.");
    cy.get('[data-cy="slot-1500"]').should("be.disabled").and("contain", "reservado");
    cy.get('[data-cy="booking-confirmation"]').should("not.be.visible");
    cy.get('[data-cy="submit-booking"]').should("be.enabled");
    cy.request("/api/reservas").its("body").should("have.length", 1)
      .its(0).its("email").should("eq", "otro@ejemplo.com");
  });

  it("no envia la reserva si falta el correo y resalta el campo", () => {
    // Arrange: horario elegido y solo el nombre completo (CP-008)
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-0900"]').click();
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");

    // Act: intentar confirmar
    cy.get('[data-cy="submit-booking"]').click();

    // Assert: mensaje visible en el campo y ningun pedido al servidor
    cy.get('[data-cy="error-email"]')
      .should("be.visible")
      .and("have.text", "Ingresa tu correo electronico para confirmar la reserva.");
    cy.get("@crearReserva.all").should("have.length", 0);
    cy.get('[data-cy="booking-confirmation"]').should("not.be.visible");
  });

  it("un sabado no ofrece horarios", () => {
    // Arrange: dia no habil segun la disponibilidad del administrador (M02)
    const sabado = proximoSabado();

    // Act: elegir el sabado en el calendario
    cy.get('[data-cy="fecha-input"]').type(sabado);

    // Assert: no se muestra ningun turno y se explica por que
    cy.get('[data-cy="agenda-vacia"]')
      .should("be.visible")
      .and("have.text", "Ese dia no hay turnos disponibles. Elegi otro dia.");
    cy.get('[data-cy^="slot-"]').should("not.exist");
  });
});
```

Navarro Matías – Reserva exitosa

Archivo: `cypress/e2e/m04-reserva-exitosa.cy.js`

```
/**
 * M04 - Booking publico | Flujos obligatorios: Seleccionar fecha y hora +
 * Completar formulario y confirmar reserva (happy path)
 * Responsable: Navarro Matias
 * Trazabilidad: AYA-M04-RF-03 y AYA-M04-RF-05 / CP-007 y CP-010
 */
import { proximoDiaHabil } from "../support/fechas";

describe("AgendaYA - M04 Booking: reserva exitosa", () => {
  const dia = proximoDiaHabil();

  beforeEach(() => {
    cy.visit("/reservar.html");
  });

  it("elige dia y horario, confirma la reserva y notifica a ambas partes", () => {
    // Arrange: elegir un dia habil futuro, un horario libre y completar los datos
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1000"]').should("be.enabled").click();
```

```
    cy.get('[data-cy="resumen-turno"]').should("contain", dia + " a las 10:00 h");
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");
    cy.get('[data-cy="email-input"]').type("juan.perez@ejemplo.com");

    // Act: confirmar la reserva
    cy.get('[data-cy="submit-booking"]').click();

    // Assert: confirmacion visible, turno ocupado en la agenda y correos disparados
    cy.get('[data-cy="booking-confirmation"]')
      .should("be.visible")
      .and("contain", "Tu reserva fue confirmada")
      .and("contain", dia);
    cy.get('[data-cy="booking-codigo"]').should("have.text", "#1");
    cy.get('[data-cy="submit-booking"]').should("be.disabled");
    cy.get('[data-cy="slot-1000"]').should("be.disabled").and("contain", "reservado");
    cy.task("leerCorreos").then((correos) => {
      expect(correos).to.contain("Para:   juan.perez@ejemplo.com");
      expect(correos).to.contain("Para:   admin@agendaya.com");
    });
  });

  it("ante varios clicks rapidos en Confirmar registra una sola reserva", () => {
    // Arrange: formulario listo para confirmar
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1100"]').click();
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");
    cy.get('[data-cy="email-input"]').type("juan.perez@ejemplo.com");

    // Act: tres clicks seguidos sobre el boton (force: el boton ya se deshabilita)
    cy.get('[data-cy="submit-booking"]').click().click({ force: true }).click({ force: true });

    // Assert: la reserva se confirma y en el sistema hay un unico registro
    cy.get('[data-cy="booking-confirmation"]').should("be.visible");
    cy.request("/api/reservas").its("body").should("have.length", 1);
  });
});
```

3.3 Ejecución y evidencia

Ejecución con `npm run test:e2e`, que levanta el servidor y corre `npx cypress run` en modo headless:

```
  (Run Finished)


       Spec                                              Tests  Passing  Failing  Pending  Skipped
  +----------------------------------------------------------------------------------------------------+
  | OK  m01-login-logout.cy.js                  00:06        2        2        -        -        - |
  +----------------------------------------------------------------------------------------------------+
  | OK  m01-registro-errores.cy.js              00:04        2        2        -        -        - |
  +----------------------------------------------------------------------------------------------------+
  | OK  m01-registro-exitoso.cy.js              00:03        1        1        -        -        - |
  +----------------------------------------------------------------------------------------------------+
  | OK  m04-reserva-errores.cy.js               00:08        3        3        -        -        - |
  +----------------------------------------------------------------------------------------------------+
  | OK  m04-reserva-exitosa.cy.js               00:07        2        2        -        -        - |
  +----------------------------------------------------------------------------------------------------+
    OK  All specs passed!                       00:30       10       10        -        -        -
```

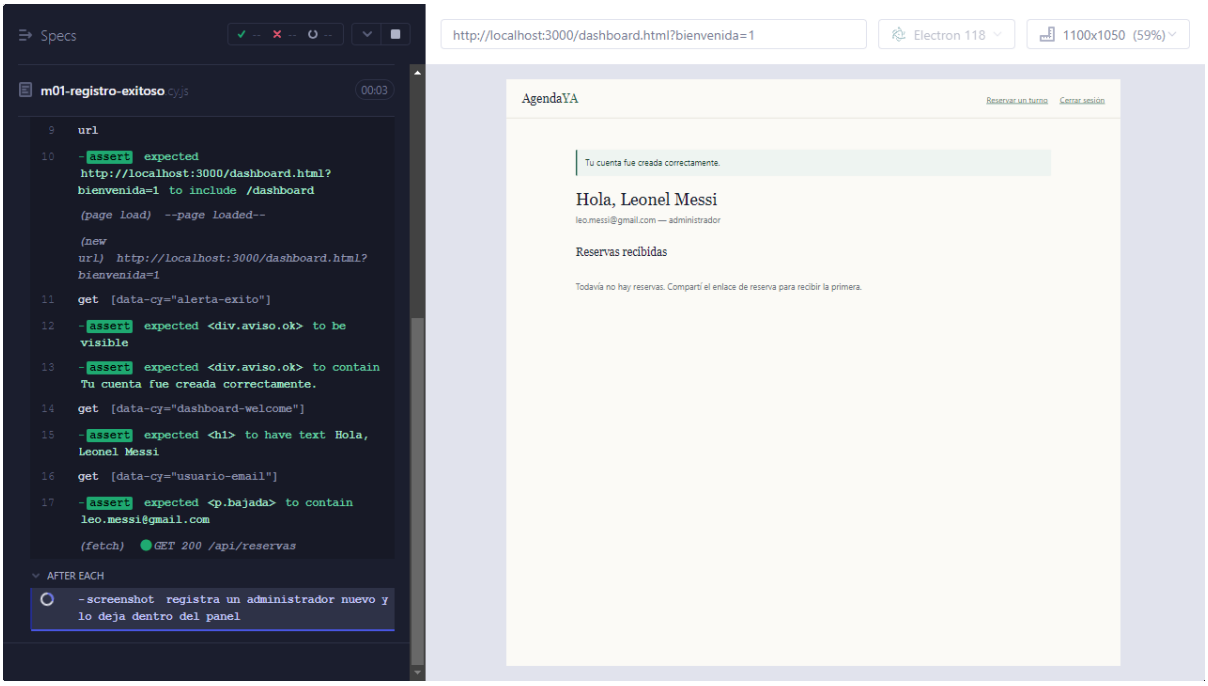


Figura 6 – m01-registro-exitoso (Santino): registro y acceso al panel.

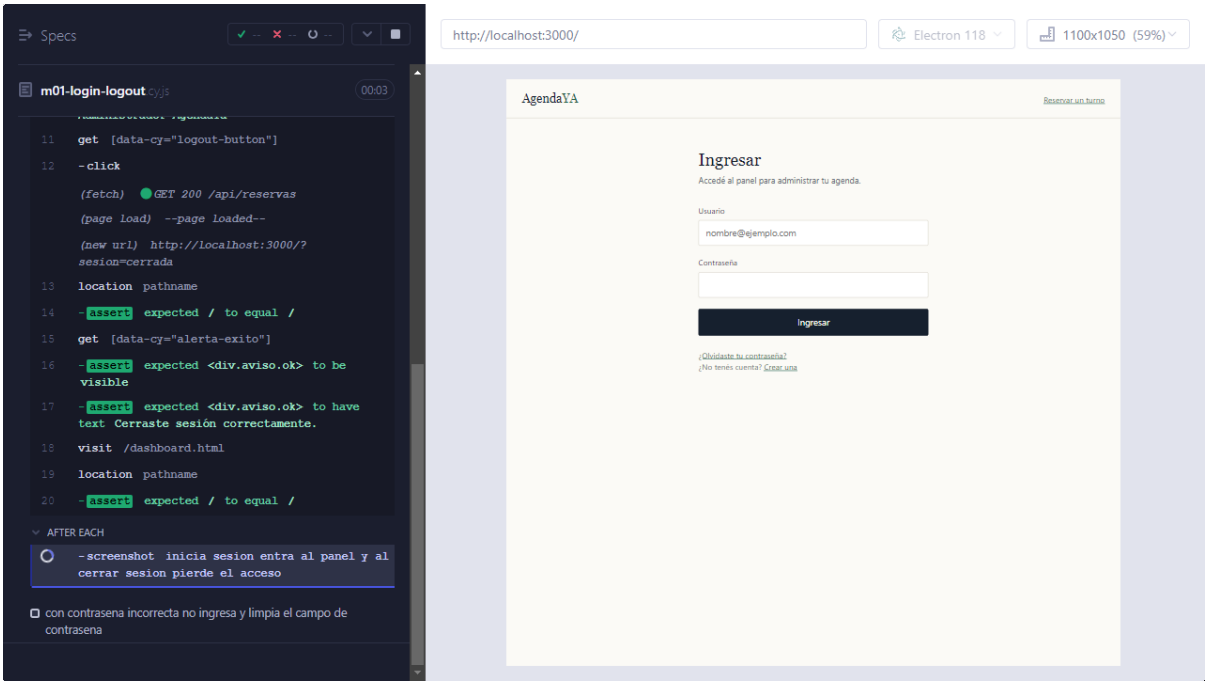


Figura 7 – m01-login-logout (Santino): tras el logout, el panel redirige al login.

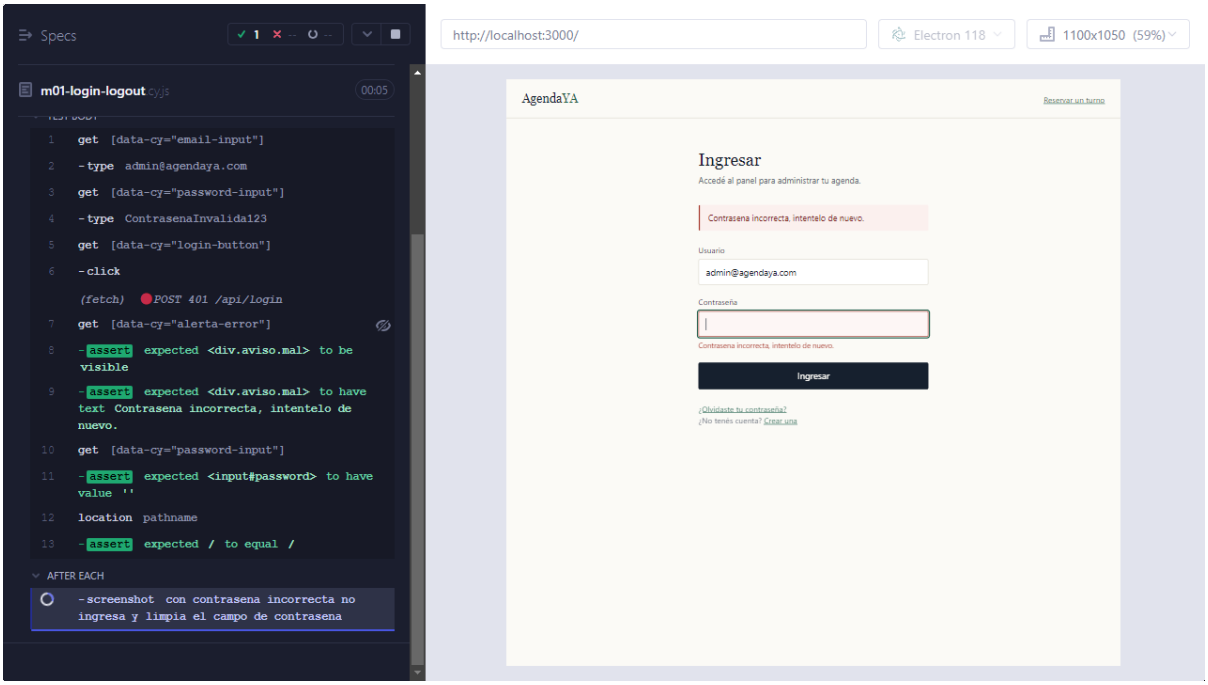


Figura 8 – m01-login-logout (Santino): contraseña incorrecta.

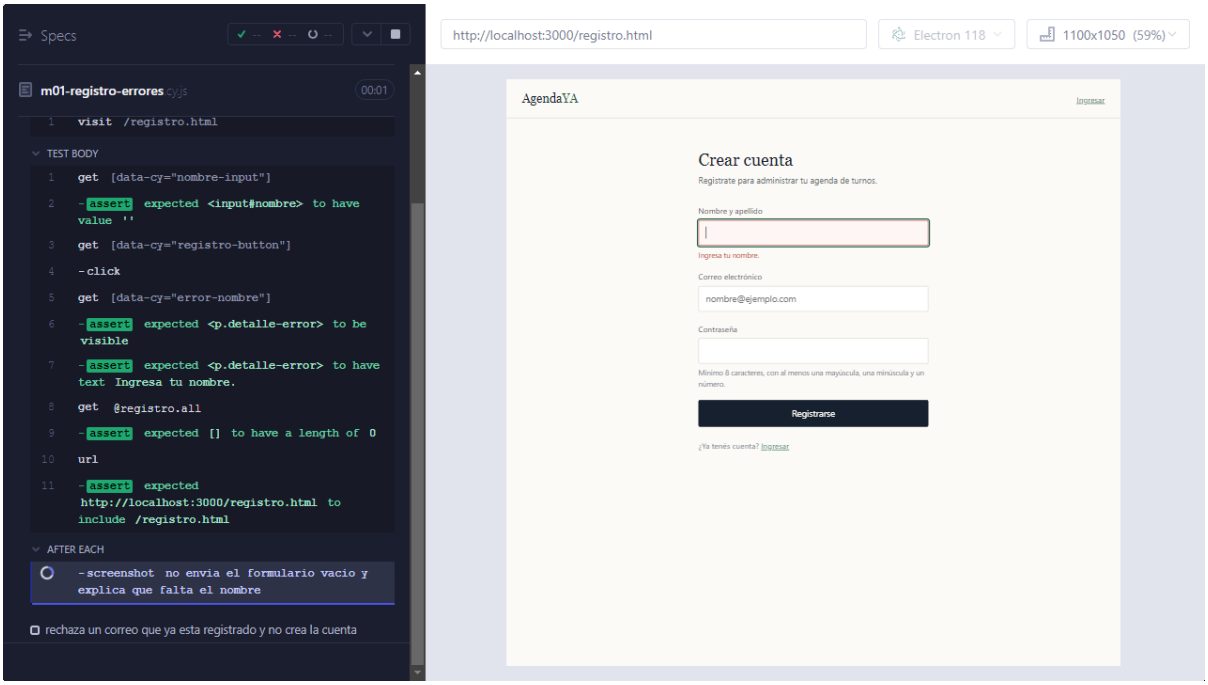


Figura 9 – m01-registro-errores (Máximo): formulario vacío, sin pedidos al servidor.

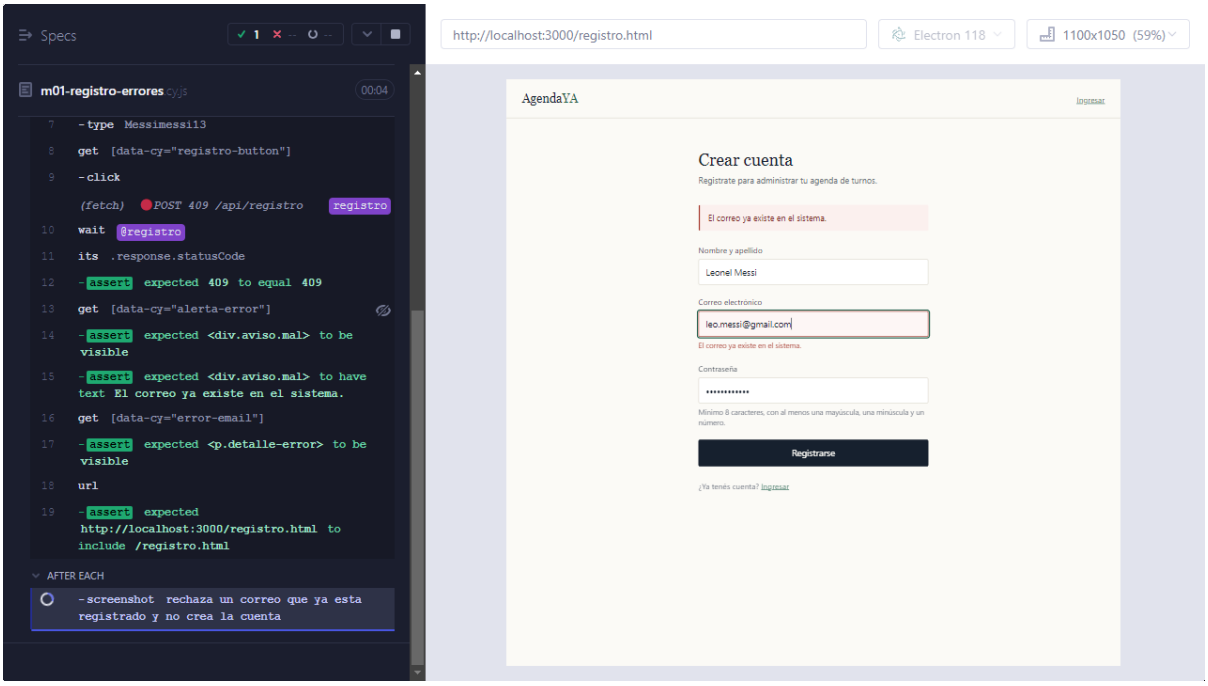


Figura 10 – m01-registro-errores (Máximo): correo ya registrado (409).

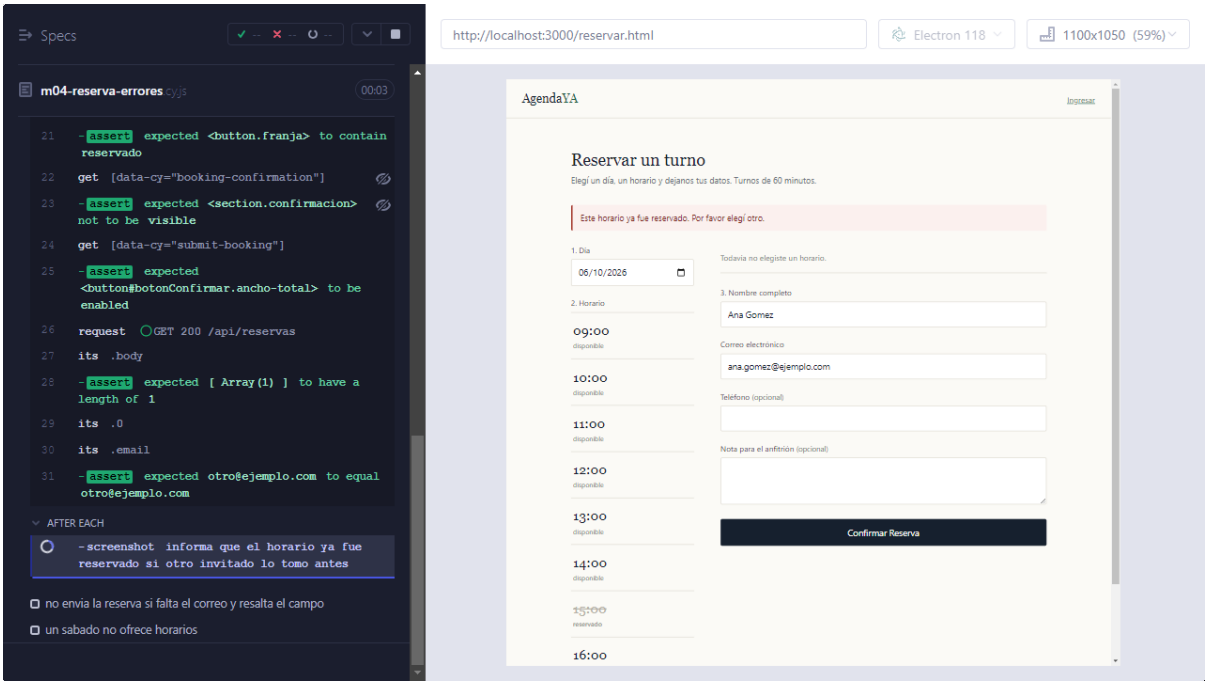


Figura 11 – m04-reserva-errores (Máximo): horario tomado por otro invitado.

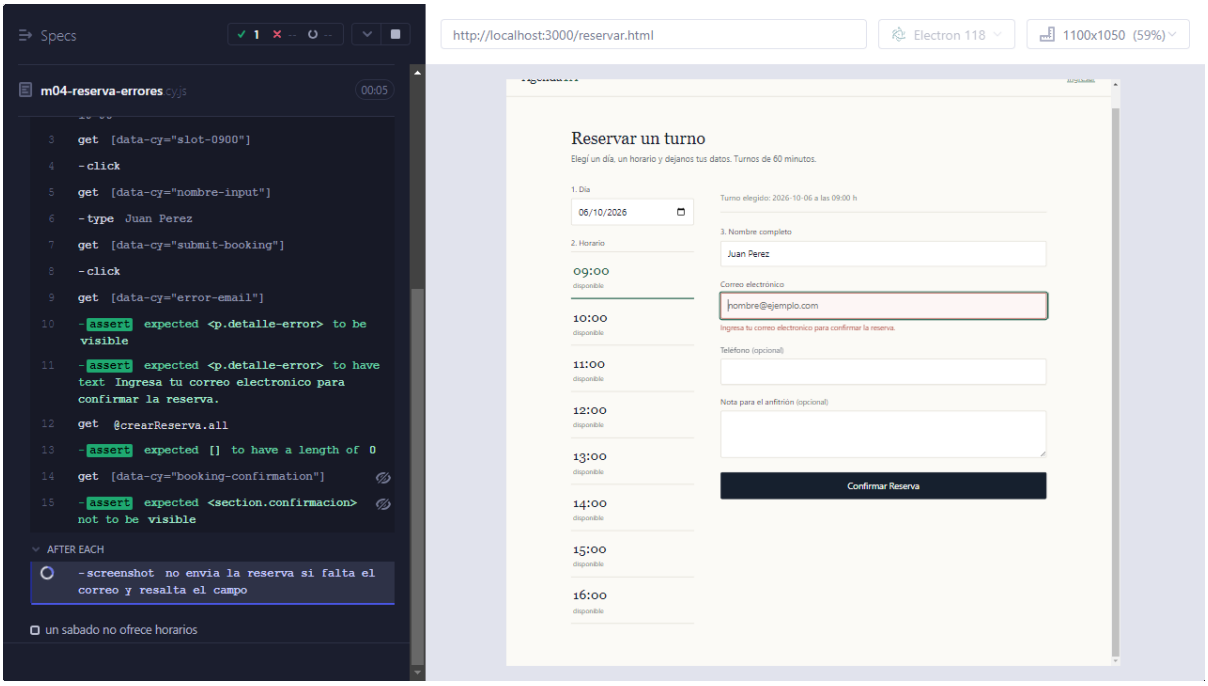


Figura 12 – m04-reserva-errores (Máximo): falta el correo.

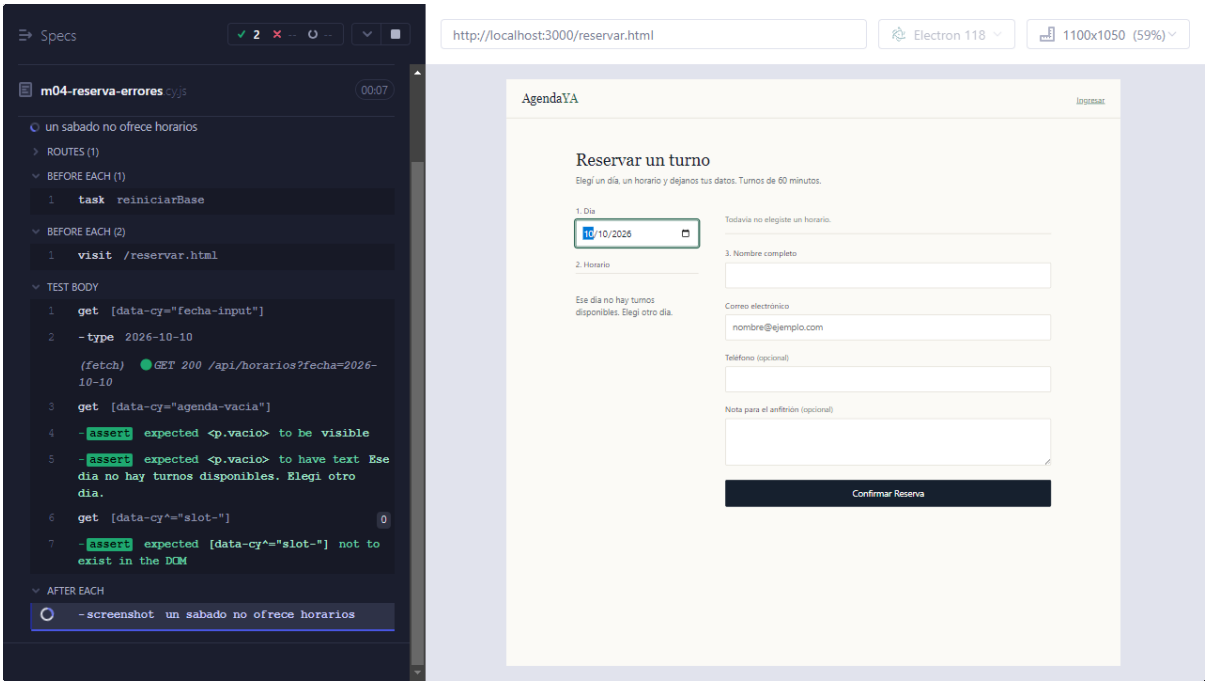


Figura 13 – m04-reserva-errores (Máximo): día no hábil.

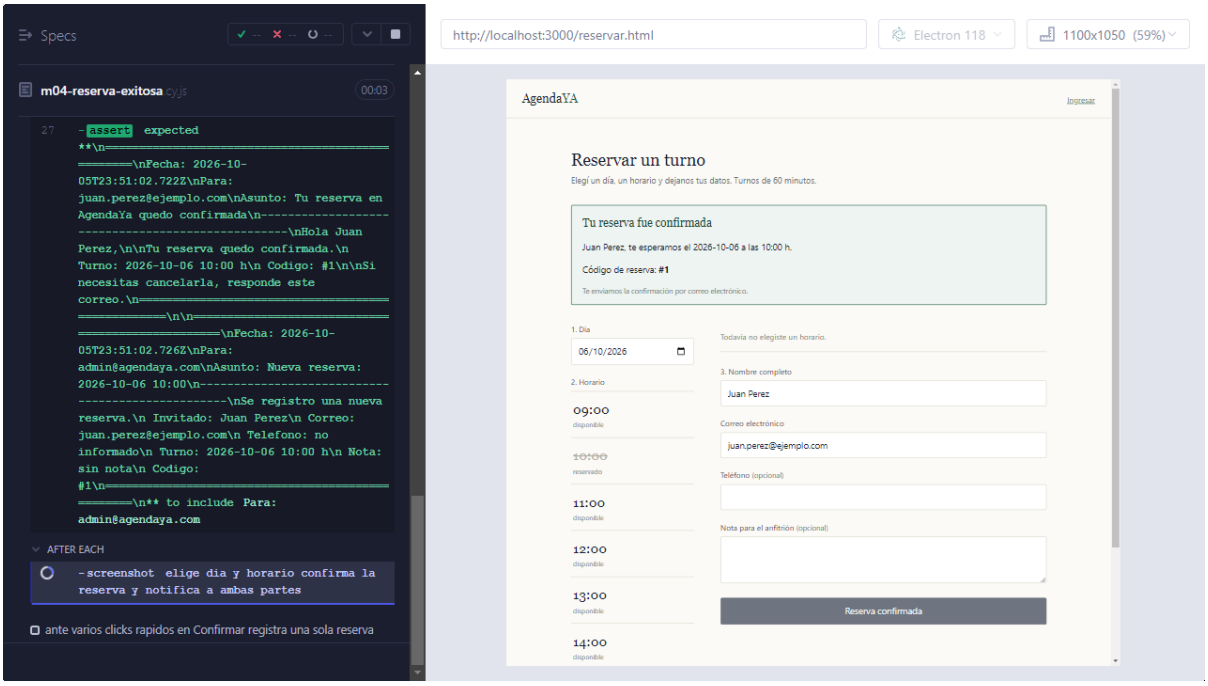


Figura 14 – m04-reserva-exitosa (Matías): reserva confirmada y correos verificados.

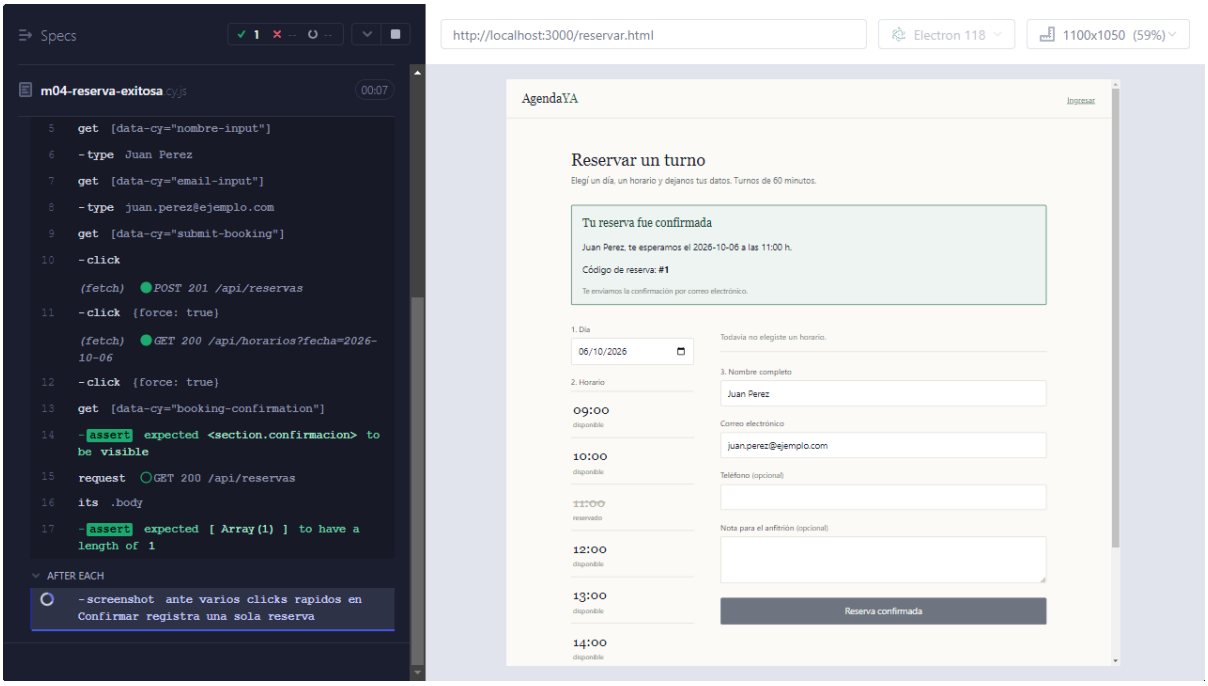


Figura 15 – m04-reserva-exitosa (Matías): varios clicks generan una única reserva.

3.4 Análisis de resultados

Resultado final: 10 tests, 10 aprobados.

Test que falló y decisión tomada. En la primera ejecución fallaron los 5 tests de M04, todos por la misma causa:

```
AssertionError: Timed out retrying after 4000ms: Expected to find element:
`[data-cy="slot-1000"]`, but never found it.
    at Context.eval (webpack://agendaya-tp6-grupo4/./cypress/e2e/m04-reserva-exitosa.cy.js:44:7)
```

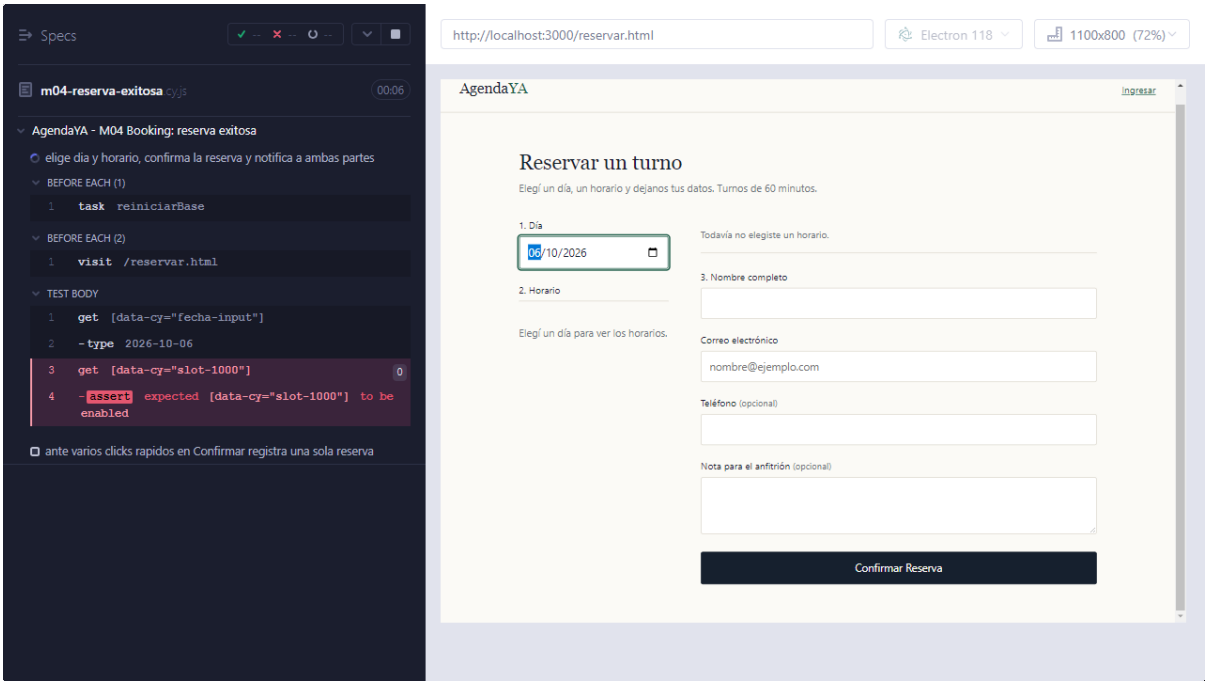


Figura 16 – Primera corrida: la fecha está cargada, pero la agenda no se actualizó.

La captura muestra la fecha cargada en el campo y la agenda vacía, con el texto "Elegí un día para ver los horarios". La pantalla cargaba la agenda solo con el evento `change` del `<input type="date">`, y `cy.type()` dispara el evento `input`. El equipo decidió **corregir el frontend** y no el test: el defecto no es exclusivo de Cypress, porque según el navegador y la forma de cargar la fecha (teclado, autocompletado) se dispara uno u otro evento. La pantalla ahora escucha ambos eventos y evita cargar dos veces la misma fecha. Con ese cambio, los 10 tests pasan.

Otras decisiones. El test de clicks múltiples verifica en el servidor (`/api/reservas`) que exista un único registro, no solo que el botón se deshabilite. Así cubre las dos barreras contra la doble reserva que se implementaron en el TP5: la bandera en el navegador y la validación en el servidor.

4. Tarea C – Tests unitarios con IA

4.1 Organización

Se usó Jest. Cada integrante tiene sus archivos en `tests/unit/`, con al menos dos funciones y casos normales, de borde e inválidos. Las funciones que dependen de la hora reciben `ahora` como parámetro y los tests usan fechas fijas.

Integrante	Archivo	Funciones	Tests
Barchiesi Santino	m01-email.test.js	normalizarEmail, esEmailValido, validarLogin	8
Barchiesi Santino	m01-seguridad.test.js	hashearPassword / verificarPassword, crearTokenRecuperacion / tokenVigente	8
Estudillo Máximo	m01-password-registro.test.js	evaluarPassword, validarRegistro	8
Estudillo Máximo	m04-validacion-conflictos.test.js	validarDatosReserva, detectarConflicto, evaluarReserva	10
Navarro Matías	m04-slots-antelacion.test.js	calcularSlots, cumpleAntelacion, esDiaHabil, agendaDelDia	11

4.2 Uso de IA: proceso general

Herramienta: Claude Code (Anthropic), un agente que trabaja en la terminal sobre el repositorio: lee archivos, escribe código y ejecuta comandos (`npm`, `jest`, `cypress`).

Prompt utilizado. Se le dio a la herramienta el repositorio del TP5, el enunciado del TP6 y el caso de estudio, con la consigna:

> "Necesito que me ayudes a realizar el trabajo práctico N°6, podés usar como base lo que construimos en el TP5. Primero entendé el problema, leé todo el contexto y pensá como un software engineer."

A partir de la sección 7 del enunciado, la herramienta propuso las funciones a testear y los archivos. Antes de escribir los tests tuvo que extraer la lógica del TP5 a funciones puras, porque en el TP5 no había nada que se pudiera probar de forma aislada. Para cada bloque se incluye además un **prompt específico equivalente**, que permite reproducir el bloque con cualquier herramienta.

Output generado. Es el código de los commits "Extraer la logica de negocio…" y "Tests unitarios con Jest…" (43 tests en la versión generada). El código final se transcribe en la sección 4.3.

Problemas encontrados en la revisión:

Qué pasó	Decisión
Los 43 tests unitarios pasaron en la primera corrida.	Un test que nunca falló no demuestra nada. Se introdujeron a mano dos defectos de borde en reservas.js: la antelación con > en lugar de >= (falló 1 test) y turnos que se pasan del horario de fin (fallaron 2). Después se restauró el código.
En un test E2E de login, la herramienta escribió "Hola, Administrador AgendaYA".replace("YA", "Ya") para que coincidiera con el nombre precargado.	Se reemplazó por el texto literal. Un test que acomoda el valor esperado esconde lo que verifica.

Qué pasó	Decisión
Un comentario // Assert decía que se verificaba el texto "Procesando reserva…", pero no había ninguna aserción sobre eso.	Se corrigió el comentario para que describa lo que realmente se verifica.
La pantalla de reserva generada escuchaba solo el evento change de la fecha (lo detectó Cypress).	Se corrigió el frontend (ver 3.4).

Lo que la herramienta no pudo resolver sola: decisiones de negocio que los requerimientos dejaban abiertas. Las tomó y las dejó explícitas en el código: las reglas de la contraseña segura (8+ caracteres, mayúscula, minúscula y número), que con exactamente 2 horas de anticipación sí se pueda reservar, que el enlace de recuperación deje de valer en el instante del vencimiento, y una disponibilidad fija, porque la define otro módulo (M02).

4.3 Tests y uso de IA por integrante

Barchiesi Santino

Archivo: `tests/unit/m01-email.test.js`

```
/**
 * M01 - Validacion de email
 * Responsable: Barchiesi Santino
 * Funciones: normalizarEmail, esEmailValido, validarLogin
 */
const { normalizarEmail, esEmailValido, validarLogin } = require("../../src/logica/validaciones");

describe("normalizarEmail", () => {
  it("quita espacios y pasa a minusculas (caso normal)", () => {
    expect(normalizarEmail("  Admin@AgendaYA.com ")).toBe("admin@agendaya.com");
  });

  it("devuelve texto vacio para null o undefined (caso borde)", () => {
    expect(normalizarEmail(null)).toBe("");
    expect(normalizarEmail(undefined)).toBe("");
  });
});

describe("esEmailValido", () => {
  it("acepta un email con usuario, dominio y extension (caso normal)", () => {
    expect(esEmailValido("leo.messi@gmail.com")).toBe(true);
  });

  it("rechaza emails sin @, sin dominio o sin extension (casos invalidos)", () => {
    expect(esEmailValido("leo.messi.gmail.com")).toBe(false);
    expect(esEmailValido("leo@")).toBe(false);
    expect(esEmailValido("leo@gmail")).toBe(false);
  });

  it("rechaza espacios internos y extensiones de una sola letra (casos borde)", () => {
    expect(esEmailValido("leo messi@gmail.com")).toBe(false);
    expect(esEmailValido("leo@gmail.c")).toBe(false);
  });

  it("rechaza valores que no son texto (caso invalido)", () => {
    expect(esEmailValido(undefined)).toBe(false);
    expect(esEmailValido(12345)).toBe(false);
  });
});

describe("validarLogin", () => {
  it("con usuario y contrasena completos no devuelve error (caso normal)", () => {
    expect(validarLogin({ email: "admin@agendaya.com", password: "Contrasena123!" })).toBeNull();
  });
```

```
  it("con el usuario vacio marca el campo email (caso invalido)", () => {
    expect(validarLogin({ email: "   ", password: "Contrasena123!" })).toEqual({
      campo: "email",
      mensaje: "Ingresa tu usuario.",
    });
  });
});
```

Archivo: `tests/unit/m01-seguridad.test.js`

```
/**
 * M01 - Contrasenas con hash y enlace de recuperacion
 * Responsable: Barchiesi Santino
 * Funciones: hashearPassword, verificarPassword, crearTokenRecuperacion, tokenVigente
 */
const {
  hashearPassword,
  verificarPassword,
  crearTokenRecuperacion,
  tokenVigente,
} = require("../../server/seguridad");

describe("hashearPassword / verificarPassword", () => {
  it("el hash no contiene la contrasena y la verificacion la reconoce (caso normal)", () => {
    const hash = hashearPassword("Contrasena123!");
    expect(hash).not.toContain("Contrasena123!");
    expect(verificarPassword("Contrasena123!", hash)).toBe(true);
  });

  it("rechaza una contrasena distinta, aunque difiera en una sola letra (caso invalido)", () => {
    const hash = hashearPassword("Contrasena123!");
    expect(verificarPassword("contrasena123!", hash)).toBe(false);
  });

  it("la misma contrasena genera hashes distintos por la sal aleatoria (caso borde)", () => {
    expect(hashearPassword("Messimessi12")).not.toBe(hashearPassword("Messimessi12"));
  });

  it("un hash guardado con formato invalido devuelve false en lugar de romper", () => {
    expect(verificarPassword("Contrasena123!", "no-es-un-hash")).toBe(false);
    expect(verificarPassword("Contrasena123!", "sal:abc")).toBe(false);
    expect(verificarPassword("Contrasena123!", undefined)).toBe(false);
  });

  it("no permite hashear una contrasena vacia", () => {
    expect(() => hashearPassword("")).toThrow("no puede estar vacia");
  });
});

describe("crearTokenRecuperacion / tokenVigente", () => {
  const ahora = new Date("2026-10-05T12:00:00.000Z");

  it("un enlace recien creado vence a los 30 minutos y esta vigente (caso normal)", () => {
    const registro = crearTokenRecuperacion("hola.comova@agendaya.com", ahora);
    expect(registro.expiraEn).toBe("2026-10-05T12:30:00.000Z");
    expect(registro.token).toMatch(/^[0-9a-f]{48}$/);
    expect(tokenVigente(registro, ahora)).toBe(true);
  });

  it("vale un milisegundo antes del vencimiento y ya no vale en el instante exacto (caso borde)", () => {
    const registro = { expiraEn: "2026-10-05T12:30:00.000Z" };
    expect(tokenVigente(registro, new Date("2026-10-05T12:29:59.999Z"))).toBe(true);
    expect(tokenVigente(registro, new Date("2026-10-05T12:30:00.000Z"))).toBe(false);
  });

  it("un registro inexistente o con fecha ilegible no es vigente (caso invalido)", () => {
    expect(tokenVigente(null, ahora)).toBe(false);
    expect(tokenVigente({ expiraEn: "manana" }, ahora)).toBe(false);
  });
});
```

Prompt específico equivalente:

> "Tengo un módulo JavaScript con normalizarEmail(valor) (email sin espacios y en minúsculas, o "" si es null/undefined), esEmailValido(email) (forma usuario@dominio.ext con extensión de 2+ letras) y validarLogin({email, password}) (null o el primer error {campo, mensaje}). También hashearPassword/verificarPassword con scrypt y sal aleatoria, y tokenVigente(registro, ahora) para un enlace que vence a los 30 minutos. [código]. Generá tests con Jest con caso normal, casos borde e inválidos. Usá fechas fijas."

Output generado: el archivo de la versión inicial (commit "Tests unitarios con Jest…").

Modificaciones: Ninguna sobre el código generado. Se reasignó la autoría de m01-seguridad.test.js al reorganizarse el equipo.

Evaluación crítica:

- **Correcto y completo:** cubre los tres tipos de caso en cada función. Los tests del token usan fechas fijas y prueban el borde exacto (un milisegundo antes del vencimiento y el instante del vencimiento), que es donde suelen aparecer los errores de < contra <=.
- **Aporte no pedido:** el caso "hash guardado con formato inválido" llevó a corregir la función del TP5, que en ese caso lanzaba una excepción en lugar de rechazar el ingreso.
- **Limitaciones:** `validarLogin` tiene solo 2 casos y no prueba la contraseña vacía. `esEmailValido` es una expresión regular simple: los tests verifican esa regla, no la validez real según el estándar (por ejemplo, dominios con acentos). Los tests de hash usan scrypt real, por lo que son los más lentos de la suite.

Estudillo Máximo

Archivo: `tests/unit/m01-password-registro.test.js`

```
/**
 * M01 - Contrasena segura y validacion del registro
 * Responsable: Estudillo Maximo
 * Funciones: evaluarPassword, validarRegistro
 */
const { evaluarPassword, validarRegistro } = require("../../src/logica/validaciones");

describe("evaluarPassword", () => {
  it("acepta una contrasena con 8+ caracteres, mayuscula, minuscula y numero (caso normal)", () => {
    expect(evaluarPassword("Messimessi12")).toEqual({ valida: true, errores: [] });
  });

  it("acepta exactamente 8 caracteres y rechaza 7 (caso borde del largo minimo)", () => {
    expect(evaluarPassword("Abcdef12").valida).toBe(true);
    expect(evaluarPassword("Abcde12")).toEqual({
      valida: false,
      errores: ["Debe tener al menos 8 caracteres."],
    });
  });

  it("informa cada regla incumplida por separado (caso invalido)", () => {
    const resultado = evaluarPassword("messimessi");
    expect(resultado.valida).toBe(false);
    expect(resultado.errores).toEqual([
      "Debe incluir al menos una mayuscula.",
      "Debe incluir al menos un numero.",
    ]);
  });

  it("trata una contrasena vacia o que no es texto como invalida sin lanzar error", () => {
    expect(evaluarPassword("").valida).toBe(false);
    expect(evaluarPassword(undefined).errores).toHaveLength(4);
  });
});
```

```
describe("validarRegistro", () => {
  const valido = { nombre: "Leonel Messi", email: "leo.messi@gmail.com", password: "Messimessi12" };

  it("con todos los datos validos no devuelve error (caso normal)", () => {
    expect(validarRegistro(valido)).toBeNull();
  });

  it("un nombre con solo espacios cuenta como vacio (caso borde)", () => {
    expect(validarRegistro({ ...valido, nombre: "   " })).toEqual({
      campo: "nombre",
      mensaje: "Ingresa tu nombre.",
    });
  });

  it("con la contrasena debil marca el campo password e incluye el motivo (caso invalido)", () => {
    const error = validarRegistro({ ...valido, password: "messi" });
    expect(error.campo).toBe("password");
    expect(error.mensaje).toContain("La contrasena no es segura.");
    expect(error.mensaje).toContain("al menos 8 caracteres");
  });

  it("informa primero el campo que aparece antes en el formulario", () => {
    expect(validarRegistro({ nombre: "", email: "mal", password: "" }).campo).toBe("nombre");
    expect(validarRegistro({ nombre: "Leo", email: "mal", password: "" }).campo).toBe("email");
  });
});
```

Archivo: `tests/unit/m04-validacion-conflictos.test.js`

```
/**
 * M04 - Datos obligatorios de la reserva y conflictos de turno
 * Responsable: Estudillo Maximo
 * Funciones: validarDatosReserva, detectarConflicto, evaluarReserva
 */
const {
  validarDatosReserva,
  detectarConflicto,
  evaluarReserva,
  MENSAJES,
} = require("../../src/logica/reservas");

const solicitud = {
  fecha: "2026-10-13",
  hora: "10:00",
  nombre: "Juan Perez",
  email: "juan.perez@ejemplo.com",
};

describe("validarDatosReserva", () => {
  it("con fecha, hora, nombre y correo validos no devuelve error (caso normal)", () => {
    expect(validarDatosReserva(solicitud)).toBeNull();
  });

  it("sin correo marca el campo email con el mensaje de CP-008 (caso invalido)", () => {
    expect(validarDatosReserva({ ...solicitud, email: "" })).toEqual({
      campo: "email",
      mensaje: "Ingresa tu correo electronico para confirmar la reserva.",
    });
  });

  it("distingue un correo vacio de uno mal escrito (caso borde)", () => {
    expect(validarDatosReserva({ ...solicitud, email: "juan.perez" }).mensaje).toBe(
      "Ingresa un correo electronico valido."
    );
  });

  it("sin horario elegido pide elegir uno antes que los datos personales", () => {
    expect(validarDatosReserva({ ...solicitud, hora: null, nombre: "" }).campo).toBe("hora");
  });
});

describe("detectarConflicto", () => {
  const existentes = [
    { id: 1, fecha: "2026-10-13", hora: "10:00", email: "otro@ejemplo.com", estado: "Confirmada" },
  ];
```

```
  it("un turno libre no tiene conflicto (caso normal)", () => {
    expect(detectarConflicto(existentes, { ...solicitud, hora: "11:00" })).toBeNull();
  });

  it("un turno tomado por otro invitado devuelve 'ocupado' con el mensaje del enunciado", () => {
    expect(detectarConflicto(existentes, solicitud)).toEqual({
      tipo: "ocupado",
      mensaje: "Este horario ya fue reservado. Por favor elegí otro.",
    });
  });

  it("el mismo invitado reenviando su reserva es 'duplicada', aunque cambie mayusculas (CP-010)", () => {
    const repetida = { ...solicitud, email: "  OTRO@ejemplo.com " };
    expect(detectarConflicto(existentes, repetida).tipo).toBe("duplicada");
  });

  it("una reserva cancelada libera el turno (caso borde)", () => {
    const cancelada = [{ ...existentes[0], estado: "Cancelada" }];
    expect(detectarConflicto(cancelada, solicitud)).toBeNull();
  });
});

describe("evaluarReserva (regla completa que aplica el servidor)", () => {
  const ahora = new Date(2026, 9, 12, 9, 0); // lunes 12/10/2026 09:00

  it("rechaza con 400 un turno de hoy sin la antelacion minima de 2 horas", () => {
    const rechazo = evaluarReserva({ ...solicitud, fecha: "2026-10-12", hora: "10:00" }, [], ahora);
    expect(rechazo.codigo).toBe(400);
    expect(rechazo.mensaje).toBe("Los turnos se reservan con al menos 2 horas de anticipacion.");
  });

  it("rechaza con 409 un turno ocupado y acepta (null) uno libre", () => {
    const ocupado = [{ fecha: "2026-10-13", hora: "10:00", email: "otro@ejemplo.com", estado: "Confirmada"
}];
    expect(evaluarReserva(solicitud, ocupado, ahora)).toMatchObject({ codigo: 409, mensaje: MENSAJES.ocupado
});
    expect(evaluarReserva(solicitud, [], ahora)).toBeNull();
  });
});
```

Prompt específico equivalente:

> "Tengo evaluarPassword(password) que devuelve {valida, errores} (mínimo 8 caracteres, mayúscula, minúscula y número) y validarRegistro, que devuelve el primer error en el orden nombre → email → password. También validarDatosReserva y detectarConflicto(reservas, solicitud), que devuelve null, {tipo: "duplicada"} si el mismo email ya reservó ese turno u {tipo: "ocupado"} si lo reservó otra persona; las canceladas no cuentan. [código]. Generá tests con Jest con caso normal, borde (7 y 8 caracteres, mayúsculas en el email, reserva cancelada) e inválido."

Output generado: el archivo de la versión inicial (commit "Tests unitarios con Jest…").

Modificaciones: Ninguna sobre el código generado. Se reasignó la autoría de m04-validacion-conflictos.test.js al reorganizarse el equipo.

Evaluación crítica:

- **Correcto y completo:** prueba el borde exacto del largo (7 caracteres rechazado, 8 aceptado) y verifica la lista completa de errores, no solo `valida: false`. En conflictos distingue "duplicada" (doble envío del mismo invitado, CP-010) de "ocupado" (otro invitado) y usa el mensaje literal del enunciado.

- **Limitación importante:** las reglas de contraseña que verifican estos tests las decidió la herramienta, porque el requerimiento no las definía. Los tests confirman que el código cumple esa regla, no que la regla sea la correcta.
- **Limitación de diseño:** los tests de `evaluarReserva` usan la disponibilidad por defecto del sistema (lunes a viernes, 9 a 17 h). Si M02 cambiara esa configuración, fallarían aunque la lógica siguiera bien. Habría sido mejor pasarles una configuración propia, como hacen los tests de Matías.

Navarro Matías

Archivo: `tests/unit/m04-slots-antelacion.test.js`

```
/**
 * M04 - Calculo de turnos y antelacion minima
 * Responsable: Navarro Matias
 * Funciones: calcularSlots, cumpleAntelacion, esDiaHabil, agendaDelDia
 */
const { calcularSlots, cumpleAntelacion, esDiaHabil, agendaDelDia } = require("../../src/logica/reservas");

describe("calcularSlots", () => {
  it("genera un turno por hora entre el inicio y el fin (caso normal)", () => {
    expect(calcularSlots("09:00", "12:00", 60)).toEqual(["09:00", "10:00", "11:00"]);
  });

  it("no ofrece un turno que terminaria despues del horario de fin (caso borde)", () => {
    expect(calcularSlots("09:00", "10:30", 60)).toEqual(["09:00"]);
    expect(calcularSlots("09:00", "10:30", 45)).toEqual(["09:00", "09:45"]);
  });

  it("devuelve una lista vacia si el inicio y el fin son iguales o la duracion no entra", () => {
    expect(calcularSlots("09:00", "09:00", 30)).toEqual([]);
    expect(calcularSlots("09:00", "09:30", 60)).toEqual([]);
  });

  it("lanza un error ante una duracion o una hora invalidas (caso invalido)", () => {
    expect(() => calcularSlots("09:00", "12:00", 0)).toThrow("entero positivo");
    expect(() => calcularSlots("09:00", "12:00", -30)).toThrow("entero positivo");
    expect(() => calcularSlots("9am", "12:00", 60)).toThrow("Hora invalida");
  });
});

describe("cumpleAntelacion", () => {
  const ahora = new Date(2026, 9, 12, 8, 0); // lunes 12/10/2026 08:00 (hora local)

  it("acepta un turno con mas de 2 horas de anticipacion (caso normal)", () => {
    expect(cumpleAntelacion(new Date(2026, 9, 12, 11, 0), ahora, 2)).toBe(true);
  });

  it("acepta exactamente 2 horas y rechaza 1 minuto menos (caso borde)", () => {
    expect(cumpleAntelacion(new Date(2026, 9, 12, 10, 0), ahora, 2)).toBe(true);
    expect(cumpleAntelacion(new Date(2026, 9, 12, 9, 59), ahora, 2)).toBe(false);
  });

  it("rechaza turnos pasados o en el instante actual, aun sin antelacion exigida (caso invalido)", () => {
    expect(cumpleAntelacion(new Date(2026, 9, 11, 10, 0), ahora, 2)).toBe(false);
    expect(cumpleAntelacion(ahora, ahora, 0)).toBe(false);
  });
});

describe("esDiaHabil", () => {
  const config = { diasHabiles: [1, 2, 3, 4, 5], diasBloqueados: ["2026-10-12"] };

  it("un dia de semana sin bloqueo es habil y un sabado no", () => {
    expect(esDiaHabil("2026-10-13", config)).toBe(true); // martes
    expect(esDiaHabil("2026-10-17", config)).toBe(false); // sabado
  });

  it("un dia bloqueado por el administrador o una fecha imposible no son habiles", () => {
    expect(esDiaHabil("2026-10-12", config)).toBe(false); // lunes bloqueado
    expect(esDiaHabil("2026-02-30", config)).toBe(false); // no existe
  });
```

```
});

// Agregado tras la revision critica: el reporte de cobertura mostraba que
// agendaDelDia (la funcion que combina todas las reglas anteriores) no tenia
// tests unitarios y solo quedaba cubierta por los E2E.
describe("agendaDelDia", () => {
  const config = {
    diasHabiles: [1, 2, 3, 4, 5],
    inicio: "09:00",
    fin: "13:00",
    duracionMin: 60,
    antelacionMinHoras: 2,
    diasBloqueados: [],
  };
  const ahora = new Date(2026, 9, 12, 9, 30); // lunes 12/10/2026 09:30

  it("marca cada turno de hoy con el motivo por el que no se puede tomar", () => {
    const reservas = [{ fecha: "2026-10-12", hora: "12:00", email: "x@y.com", estado: "Confirmada" }];
    expect(agendaDelDia("2026-10-12", reservas, ahora, config)).toEqual({
      fecha: "2026-10-12",
      habil: true,
      slots: [
        { hora: "09:00", disponible: false, motivo: "pasado" },
        { hora: "10:00", disponible: false, motivo: "antelacion" },
        { hora: "11:00", disponible: false, motivo: "antelacion" },
        { hora: "12:00", disponible: false, motivo: "ocupado" },
      ],
    });
  });

  it("un dia no habil no ofrece turnos", () => {
    expect(agendaDelDia("2026-10-17", [], ahora, config)).toEqual({
      fecha: "2026-10-17",
      habil: false,
      slots: [],
    });
  });
});
```

Prompt específico equivalente:

> "Tengo una función JavaScript calcularSlots(inicio, fin, duracionMin) que recibe horas "HH:MM" y una duración en minutos y devuelve un array con los horarios de inicio de cada turno que entra completo en el rango; lanza un error si la duración no es un entero positivo o la hora es inválida. También cumpleAntelacion(fechaHora, ahora, horasMin). [código]. Generá al menos 5 tests con Jest que cubran: el caso normal, duración mayor al rango disponible, horarios iguales, valores inválidos y el borde exacto de 2 horas de antelación."

Output generado: el archivo de la versión inicial (commit "Tests unitarios con Jest…").

Modificaciones: Se agregaron 2 tests de `agendaDelDia`. El reporte de cobertura mostraba esa función sin tests unitarios: es la que combina todas las reglas anteriores (turno pasado, sin antelación, ocupado) y solo quedaba cubierta por los E2E. Con el agregado, la cobertura de líneas de `reservas.js` pasó del 81 % al 92 %.

Evaluación crítica:

- **Correcto y completo:** es el bloque que detectó los dos defectos introducidos a propósito (antelación con > y un turno que se pasa del horario de fin). Prueba explícitamente que no se ofrece un turno que no entra completo: entre 09:00 y 10:30, con turnos de 60 minutos, devuelve solo 09:00.
- **Buena decisión:** construye las fechas con `new Date(año, mes, día, hora)` en hora local, por lo que los tests dan el mismo resultado en cualquier zona horaria.

- **Limitaciones:** no se probó un cambio de horario de verano (Argentina no lo aplica hoy). La herramienta no detectó sola el hueco de `agendaDelDia`: apareció al revisar el reporte de cobertura.

4.4 Evidencia de ejecución

Ejecución con `npx jest --verbose --coverage`:

```
PASS tests/unit/m04-validacion-conflictos.test.js
  validarDatosReserva
    √ con fecha, hora, nombre y correo validos no devuelve error (caso normal) (15 ms)
    √ sin correo marca el campo email con el mensaje de CP-008 (caso invalido) (3 ms)
    √ distingue un correo vacio de uno mal escrito (caso borde) (1 ms)
    √ sin horario elegido pide elegir uno antes que los datos personales (1 ms)
  detectarConflicto
    √ un turno libre no tiene conflicto (caso normal) (1 ms)
    √ un turno tomado por otro invitado devuelve 'ocupado' con el mensaje del enunciado
    √ el mismo invitado reenviando su reserva es 'duplicada', aunque cambie mayusculas (CP-010) (7 ms)
    √ una reserva cancelada libera el turno (caso borde) (1 ms)
  evaluarReserva (regla completa que aplica el servidor)
    √ rechaza con 400 un turno de hoy sin la antelacion minima de 2 horas (3 ms)
    √ rechaza con 409 un turno ocupado y acepta (null) uno libre (3 ms)

PASS tests/unit/m04-slots-antelacion.test.js
  calcularSlots
    √ genera un turno por hora entre el inicio y el fin (caso normal) (20 ms)
    √ no ofrece un turno que terminaria despues del horario de fin (caso borde) (1 ms)
    √ devuelve una lista vacia si el inicio y el fin son iguales o la duracion no entra (1 ms)
    √ lanza un error ante una duracion o una hora invalidas (caso invalido) (73 ms)
  cumpleAntelacion
    √ acepta un turno con mas de 2 horas de anticipacion (caso normal) (2 ms)
    √ acepta exactamente 2 horas y rechaza 1 minuto menos (caso borde) (1 ms)
    √ rechaza turnos pasados o en el instante actual, aun sin antelacion exigida (caso invalido) (1 ms)
  esDiaHabil
    √ un dia de semana sin bloqueo es habil y un sabado no (1 ms)
    √ un dia bloqueado por el administrador o una fecha imposible no son habiles (1 ms)
  agendaDelDia
    √ marca cada turno de hoy con el motivo por el que no se puede tomar (2 ms)
    √ un dia no habil no ofrece turnos (1 ms)

PASS tests/unit/m01-password-registro.test.js
  evaluarPassword
    √ acepta una contrasena con 8+ caracteres, mayuscula, minuscula y numero (caso normal) (3 ms)
    √ acepta exactamente 8 caracteres y rechaza 7 (caso borde del largo minimo) (2 ms)
    √ informa cada regla incumplida por separado (caso invalido) (2 ms)
    √ trata una contrasena vacia o que no es texto como invalida sin lanzar error (1 ms)
  validarRegistro
    √ con todos los datos validos no devuelve error (caso normal) (1 ms)
    √ un nombre con solo espacios cuenta como vacio (caso borde) (1 ms)
    √ con la contrasena debil marca el campo password e incluye el motivo (caso invalido) (1 ms)
    √ informa primero el campo que aparece antes en el formulario (1 ms)

PASS tests/unit/m01-email.test.js
  normalizarEmail
    √ quita espacios y pasa a minusculas (caso normal) (1 ms)
    √ devuelve texto vacio para null o undefined (caso borde) (1 ms)
  esEmailValido
    √ acepta un email con usuario, dominio y extension (caso normal) (1 ms)
    √ rechaza emails sin @, sin dominio o sin extension (casos invalidos)
    √ rechaza espacios internos y extensiones de una sola letra (casos borde) (1 ms)
    √ rechaza valores que no son texto (caso invalido)
  validarLogin
    √ con usuario y contrasena completos no devuelve error (caso normal)
    √ con el usuario vacio marca el campo email (caso invalido) (1 ms)

PASS tests/unit/m01-seguridad.test.js
  hashearPassword / verificarPassword
    √ el hash no contiene la contrasena y la verificacion la reconoce (caso normal) (294 ms)
    √ rechaza una contrasena distinta, aunque difiera en una sola letra (caso invalido) (211 ms)
    √ la misma contrasena genera hashes distintos por la sal aleatoria (caso borde) (204 ms)
    √ un hash guardado con formato invalido devuelve false en lugar de romper (2 ms)
    √ no permite hashear una contrasena vacia (34 ms)
  crearTokenRecuperacion / tokenVigente
    √ un enlace recien creado vence a los 30 minutos y esta vigente (caso normal) (4 ms)
    √ vale un milisegundo antes del vencimiento y ya no vale en el instante exacto (caso borde) (1 ms)
    √ un registro inexistente o con fecha ilegible no es vigente (caso invalido) (1 ms)
```

```
------------------|---------|----------|---------|---------|--------------------------
File              | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
------------------|---------|----------|---------|---------|--------------------------
All files         |   91.66 |       80 |   96.66 |   93.95 |
 server           |     100 |     91.3 |     100 |     100 |
  seguridad.js    |     100 |     91.3 |     100 |     100 | 37,54
 src/logica       |   90.38 |    78.03 |   96.15 |   93.02 |
  reservas.js     |   90.56 |    77.38 |   94.73 |   92.13 | 14,80,160,166,209,212,216
  validaciones.js |      90 |    79.16 |     100 |      95 | 13,81
------------------|---------|----------|---------|---------|--------------------------
Test Suites: 5 passed, 5 total
Tests:       45 passed, 45 total
Snapshots:   0 total
Time:        2.854 s, estimated 3 s
Ran all test suites.
```

Las líneas sin cubrir son de dos tipos. Algunas solo se ejecutan en el navegador (la carga del módulo con `<script>` y `fechaIso`, que calcula la fecha mínima del calendario) o son valores por defecto de parámetros opcionales. Otras son ramas de rechazo sin test unitario propio: en `validarDatosReserva`, la fecha vacía y el nombre vacío; en `evaluarReserva`, el día no hábil, la hora fuera de la agenda y el turno pasado. El día no hábil se verifica en un test E2E, pero solo en la pantalla; las demás ramas quedan como mejora pendiente para una próxima iteración.

5. Reflexión estructurada

5.1 Sobre la trazabilidad

En gran parte, sí. Los casos de prueba del TP5, derivados de los requerimientos, se pudieron convertir casi directamente en tests: los que citaban el mensaje exacto del sistema (CP-002, CP-004, CP-008) se transformaron en aserciones sin necesidad de interpretar nada. Las dificultades aparecieron donde el requerimiento dejaba una decisión abierta. AYA-M01-RF-01 pide una contraseña "que cumpla con los requisitos de seguridad" sin decir cuáles, y para escribir `evaluarPassword` hubo que definir la regla. La antelación mínima ("no se puede reservar con menos de 2 horas") no aclara si exactamente 2 horas se permite, y un test de borde obliga a elegir. Tampoco estaba documentada la dependencia de M04 con M02: sin la disponibilidad del administrador no se puede probar la selección de fecha, y hubo que fijar una configuración.

5.2 Sobre el valor de testear

Sí. Cypress detectó que la agenda no se cargaba al escribir la fecha, porque la pantalla escuchaba un solo evento. Al preparar el flujo de selección de fecha se vio que los horarios del TP5, fijos al 15/09/2026, ya habían pasado y el sistema los seguía ofreciendo, contra lo que pide el enunciado. Al diseñar el caso "hash guardado con formato inválido" se vio que la función del TP5 lanzaba una excepción en lugar de rechazar el ingreso. Al agregar los `data-cy` al panel se encontró que la lista de reservas se armaba con `innerHTML` usando datos escritos por el invitado, lo que permitía inyectar código (XSS). Ninguno de estos problemas había aparecido al ejecutar a mano los 10 casos del TP5. El testing aporta más cuando se escribe junto con el código y obliga a pensar en los bordes, no solo como verificación final.

5.3 Sobre el uso de IA

La IA fue más útil para el código fuente mínimo y el frontend (estructura, formularios con sus `data-cy`, configuración de Cypress y Jest), que es trabajo mecánico, y para enumerar casos de borde en los tests unitarios. Las limitaciones aparecieron en la calidad de los tests: escribió un test que ajustaba el valor esperado para que pasara, dejó un comentario que no coincidía con lo verificado y no detectó por sí sola una función sin tests unitarios. Tampoco podía resolver las ambigüedades de los requerimientos; las resolvió tomando decisiones que el equipo tuvo que revisar. Además, que todos los tests pasen no prueba que sean útiles: hubo que introducir defectos a propósito para comprobar que los detectaban.

6. Lecciones aprendidas

1. **Separar la lógica de la interfaz es lo que hace posible el test unitario.** En el TP5 las reglas estaban mezcladas con HTTP y con el DOM, y no había nada que probar de forma aislada. Extraerlas a funciones puras permitió 45 tests que corren en unos 3 segundos y validan el mismo código que usa la pantalla.
2. **El tiempo es una dependencia y hay que inyectarlo.** Una función que llama a `new Date()` por dentro da resultados distintos según el día. Recibir `ahora` como parámetro hizo repetibles los tests de antelación y de vencimiento, y en los E2E calcular el próximo día hábil evita que los tests venzan solos.
3. **Un test E2E que falla puede estar señalando un defecto real de la aplicación.** El fallo de Cypress con el campo de fecha parecía un problema de la herramienta, pero revelaba que la pantalla dependía de un solo evento del navegador. Antes de arreglar el test conviene preguntarse si un usuario real podría pasar por el mismo camino.
4. **Los atributos data-cy desacoplan los tests del diseño.** Cambiar textos, clases o la estructura de las pantallas no rompió ningún test, porque los selectores apuntan a atributos pensados para eso y no a la apariencia.
5. **La IA acelera, pero lo generado hay que revisarlo como código de otro.** Generó en minutos la estructura y la mayoría de los casos, pero también un test que acomodaba el resultado esperado y un comentario que no correspondía con la aserción. Sin revisión, esos tests habrían dado una falsa seguridad.
6. **Que todos los tests pasen no prueba que sirvan.** Introducir defectos a propósito (cambiar un `>=` por `>`) y ver que los tests fallan es una forma barata de comprobar que verifican algo. El reporte de cobertura, por su parte, mostró una función sin tests unitarios.
7. **Un requerimiento ambiguo se nota cuando hay que escribir la aserción.** "Requisitos de seguridad" o "menos de 2 horas" se leen bien en el documento, pero el test exige un valor concreto. Escribir los criterios de aceptación con ejemplos en el TP1 (7 caracteres: rechazado; exactamente 2 horas: aceptado) habría evitado tener que decidirlo al programar.
8. **Hay que verificar el estado del sistema, no solo la pantalla.** Los tests más valiosos comprueban también lo que no se ve: que se creó una sola reserva ante varios clicks, que se dispararon los dos correos o que el servidor respondió 409. Una pantalla correcta con un estado incorrecto sigue siendo un defecto.